From CI/CD to ACG: From Tests to Courts in Agentic Software Development

Constitutional Synchronization, Governed Effects, and Reasoning Capability Through Typed Co-Projection

Tests tell us whether predicates hold. Courts govern what follows. Constitutional synchronization tells us what the predicates are about–and governed effect boundaries decide what may become real.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Continuous integration and continuous delivery transformed software engineering by making builds, tests, packaging, and deployment checks part of the infrastructure of change rather than optional developer practices. Agentic coding changes the limiting problem. As software agents become capable of generating and modifying implementation faster than humans can directly review it, the central engineering bottleneck shifts from code production toward epistemic and constitutional control: determining what a change means, whether an implementation corresponds to its specification, whether available evidence establishes the claimed behavior, whether previous evidence remains applicable to the exact current subject, whether a transition is authorized, what concrete effect should occur, and what may truthfully be claimed afterward.

This paper introduces **Ableman Constitutional Governance (ACG)** as a software-development architecture in which a constitutionally significant subset of executable verification is promoted from **tests** into **courts**. Within the software domain considered here,

$$C_{\text{software}} \subseteq V_{\text{executable}},$$

where ordinary unit, integration, regression, determinism, packaging, performance, and recurrence tests remain part of the broader verification estate. Courts differ not because they cease to be executable verifications, but because selected predicates are bound to an exact subject, constitutional frame, proposition or obligation, evidence topology, jurisdiction, authority relationship, and typed disposition.

The paper develops a broader claim: the reasoning capability obtained from ACG does not arise from courts alone. It arises from combining constitutionally typed verification with **constitutional synchronization**, governed effects, and a richer representation of the system. **Ableman's Razor**, originally formulated as a typed grammar governing migration of claims among Specification, Configuration, and Implementation, is explicitly extended here as a representation lens for governed systems. The current integration resolves Configuration into **Configuration Manifold 1 (CM1)**, the mandatory admissible manifold; **Configuration Manifold 2 (CM2)**, the preferred admissible manifold under declared criteria; and **IC**, expanded interchangeably as **Instance Constitution** or **Instance Configuration**, the selected slice for a concrete system. **IR**, expanded interchangeably as **Instance Realization** or **Implementation Realization**, makes that selection concrete:

$$\begin{aligned} & P \\ & \to S \\ & \to CM_1 \\ & \to CM_2 \\ & \to IC \\ & \to IR \\ & \to E \\ & \to R \end{aligned} \quad .$$

This extension is not attributed to the Razor's original jurisdiction. It is accepted only insofar as the added layers yield diagnostic, governance, selection, correspondence, evidence, or revision distinctions unavailable from a less differentiated representation without equivalent structure.

This representation connects to **Dynatology**, the study of possible continuation: which transitions are conceivable, specification-valid, admissible, reachable, viable, selected, authorized, realized, observed, and established from a particular state. ACG supplies courts that adjudicate relationships and transitions among these projections.

The architecture also extends governance into consequential execution through **constitutional effect interposition**. Application logic may continue to propose familiar semantic operations while a governed effect boundary determines the admissible realization under the current Instance Constitution. The requested operation, admitted realization, and actual observed effect are therefore distinct:

$$O_r \to R_a \to E_a.$$

A governed boundary may preserve, negate, constrain, transform, defer, simulate, sandbox, or substitute a proposed effect. This is not claimed to be novel merely

as interception or mutation: reference monitors, capability systems, admission controllers, aspect-oriented programming, and effect handlers provide important precedents. The distinguishing concern is the integration of effect mediation with exact-subject binding, constitutional selection, evidentiary correspondence, typed dispositions, receipts, lineage, and standing.

Because governed effects cross explicit semantic boundaries, those same boundaries become causal observation points. Receipts can record what operation was proposed, what realization was admitted, what executor action occurred, and what post-effect evidence followed. Governance, behavioral adaptation, and debugging can therefore share one effect topology.

Documentation is similarly repositioned. Rather than remaining an independently authored and perpetually drifting artifact, documentation, glossaries, implementation codices, agent-orientation surfaces, capability references, operator explanations, and configuration views become **deterministic, zero-authority projections of the retained semantic model**. Projection fidelity does not itself establish realization fidelity; correspondence must be independently witnessed and adjudicated.

The paper defines a **reasoning-capability gain** operationally as a representational extension that, while holding the underlying observed system state fixed, enables derivation of a decision-relevant distinction, contradiction, obligation, or admissible-action restriction that the baseline representation cannot express or derive without introducing equivalent additional structure. Several development episodes illustrate this pattern: environmental extraction safety conflicted with realization fidelity; artifact reuse was separated from effectful materialization; source changes invalidated stale verification evidence; and historical verifiers were made to testify to their own supersession rather than masquerade as current authority.

The resulting software project is no longer modeled merely as source code plus tests and documentation. It becomes a **constitutionally synchronized semantic and effect state** containing specification, configuration possibilities, instance commitments, implementation bindings, evidence, courts, authority, effects, receipts, lineage, standing, and deterministic projections. ACG therefore suggests a progression from continuous integration toward **continuous constitutionalization**: not a replacement for CI/CD, but an enlargement in which verification, correspondence, adjudication, effect governance, and explanation become machine-addressable parts of the development process.

---

1. Introduction

Software engineering repeatedly advances by turning practices dependent on individual memory or discipline into infrastructure.

Version control made lineage infrastructural. Automated testing made behavioral

checking repeatable. Continuous integration made verification an expected consequence of change. Continuous delivery extended the same principle into packaging and release automation.

The transformative insight of CI/CD was therefore not the invention of testing. Tests long predated continuous integration (Beck, 2002). The important change was that verification became structurally integrated into the development process (Fowler, 2006; Duvall, Matyas, and Glover, 2007; Humble and Farley, 2010):

$$\begin{aligned} &\text{change} \\ &\rightarrow \text{build} \\ &\rightarrow \text{test} \\ &\rightarrow \text{integrate} \\ &\rightarrow \text{deliver} \end{aligned} \quad .$$

A test ceased to be merely something a developer *could* run. It became something the development system itself expected to run.

Agentic software development creates a new asymmetry.

A capable coding agent may inspect a repository, modify many files, regenerate derived artifacts, repair compilation errors, alter tests, update build infrastructure, package candidate releases, and produce persuasive explanations in a period that previously required sustained human work.

The capacity to **produce implementation** can therefore grow faster than the capacity to establish **justified belief about implementation**.

This produces a different limiting question.

Not merely:

> Can the agent write the code?

but:

> **What prevents the agent, developer, or organization from confusing successful production with justified knowledge–and justified knowledge with authority to change reality?**

Traditional CI/CD answers important questions:

- Does the source compile?
- Do the tests pass?
- Does the artifact package correctly?
- Is a build deterministic?
- Does deployment complete?
- Does a monitored service remain healthy?

Agentic development adds questions of another type:

- What does the requested operation mean?
- What system state is the claim actually about?
- Which configuration possibilities were admissible?
- Which of those possibilities did this instance actually select?
- Does the implementation correspond to that selection?
- Does the runtime consume the configuration that was generated?
- Does the observed behavior establish the modeled operation?
- Does old evidence still belong to the current subject?
- Did an environment transform a semantically significant property?
- Does an available capability have an actual realization?
- Does an evidentiary PASS confer any authority?
- If an operation is authorized, must its originally requested effect occur unchanged?
- What effect did the executor actually produce?
- What may legitimately be claimed about the resulting artifact?

These are not only behavioral questions.

They concern **meaning, possibility, correspondence, evidence, jurisdiction, authority, effect, lineage, and standing**.

Ableman Constitutional Governance addresses this problem through five related transformations:

1. **Tests become courts where constitutional significance exists.**
2. **Specification and implementation become constitutionally synchronized through typed intermediate representations and correspondence evidence.**
3. **Consequential effects become explicit governed transitions rather than implicit consequences of application control flow.**
4. **Receipts and witnesses make those transitions causally reconstructable.**
5. **Documentation becomes a deterministic projection of retained semantic state rather than a second, independently maintained ontology.**

The claim is not that ordinary software engineering lacks analogues for any individual mechanism. Formal methods, model-driven engineering, software product lines, admission control, reference monitors, capabilities, effect systems, build systems, and observability all supply important neighboring ideas.

The proposed contribution is their integration into a single **constitutional development process model** whose primary discipline is non-collapse:

what is possible
$\neq$ what was selected
$\neq$ what was implemented
$\neq$ what was observed
$\neq$ what was established .
$\neq$ what was authorized
$\neq$ what occurred
$\neq$ what has standing

---

2. From CI/CD Tests to Constitutional Courts

2.1 The verification estate remains larger than the court system

ACG does not replace conventional testing.

A mature software project still requires:

- unit tests;
- integration tests;
- regression tests;
- property tests;
- fuzz tests;
- static analysis;
- packaging and hygiene checks;
- deterministic-build checks;
- dependency checks;
- compatibility tests;
- performance tests;
- resource-envelope tests;
- environment recurrence tests.

A conventional test may be represented as:

$$T(x) \rightarrow \text{PASS}, \text{FAIL}.$$

For example:

> Given input (x), does function (f) produce expected output (y)?

That is useful verification.

It need not be constitutional.

2.2 Courts are constitutionally typed verification

A court becomes appropriate when an executable result participates in a governed proposition.

A simplified court can be represented as:

$$C = (S, F, P, E, J, A, D),$$

where:

- (S) = exact subject;
- (F) = constitutional frame;
- (P) = proposition or obligation;
- (E) = admissible evidence topology;
- (J) = jurisdiction or applicability;
- (A) = authority relationship;
- (D) = typed disposition.

The court therefore asks more than:

> Did predicate (P) evaluate successfully?

It asks:

> **For this exact subject, under this frame, using evidence admissible for this jurisdiction, what disposition is warranted, and what–if anything–may lawfully follow from it?**

Its vocabulary consequently need not be Boolean:

```
PASS
FAIL
NOT_ESTABLISHED
REFUSE
HOLD_RESOURCE_ENVELOPE
HISTORICAL_SUPERSEDED
```

Thus:

$$\begin{aligned} &\text{false} \\ &\neq \text{unknown} \\ &\neq \text{inadmissible} \\ &\neq \text{resource-incomplete} \\ &\neq \text{superseded} \end{aligned} \quad .$$

A binary testing interface can erase these distinctions.

A court preserves them.

2.3 The subset claim is software-scoped

Within the agentic software-development setting examined here:

$$\boxed{C_{\text{software}} \subseteq V_{\text{executable}}}$$

That is, courts are a constitutionally typed subset of executable verification.

The statement is deliberately scoped.

A legal court, human governance body, clinical review mechanism, or other non-software adjudicative institution need not itself be an executable software test.

The subset relationship describes the software realization of ACG, not an ontological definition of courts.

The topology is therefore:

```
EXECUTABLE VERIFICATION

 unit tests
 integration tests
 regression tests
 determinism tests
 packaging tests
 performance tests
 recurrence tests

 CONSTITUTIONAL COURTS
     realization correspondence
     evidence admissibility
     operation reachability
     receipt-before-effect
     authority traceability
     continuity
     standing boundaries
```

The compressed comparison is:

> **CI/CD operationalizes tests. ACG operationalizes constitutionally significant adjudication.**

---

3. Constitutional Synchronization and the Representation Problem

3.1 The conventional four-artifact problem

Software projects often maintain four partly independent accounts of themselves:

```
SPECIFICATION
IMPLEMENTATION
TESTS
DOCUMENTATION
```

They are usually joined through human interpretation.

A requirement is written.

A developer interprets it into code.

Tests are written against some interpretation of intended behavior.

Documentation is authored or updated separately.

Over time:

```
specification drifts
implementation drifts
tests drift
documentation drifts
```

The synchronization mechanism is primarily social:

- review;
- meetings;
- issue trackers;
- release checklists;
- institutional memory.

Agentic coding makes this increasingly fragile.

An agent may update implementation but miss documentation.

More dangerously, it may update implementation, tests, and documentation around the **same false assumption**.

The artifacts can become mutually consistent while remaining inconsistent with the actual runtime system.

The deeper problem is therefore not merely stale documentation.

It is **representational independence without machine-addressable correspondence**.

3.2 Ableman's Razor and a Declared Extension

Ableman's Razor was originally formulated as a typed grammar for claim migration across Specification, Configuration, and Implementation. Its layers are jurisdictional roles rather than a lifecycle. This paper deliberately applies that vocabulary to the representation of a governed system. That extension should not be silently attributed to the Razor's original jurisdiction.

The current corpus realization funnel makes the extension explicit:

$$
\begin{aligned}
& P \\
& \rightarrow S \\
& \rightarrow CM_1 \\
& \rightarrow CM_2 \\
& \rightarrow IC \\
& \rightarrow IR \\
& \rightarrow E \\
& \rightarrow R
\end{aligned}
\quad .
$$

Possibility P is the space of candidate problems, meanings, structures, and realizations under consideration. Specification S states types, invariants, operations, forbidden states, evidence obligations, and claim boundaries. **Configuration Manifold 1 (CM1)** contains every configuration satisfying mandatory admissibility. **Configuration Manifold 2 (CM2)** contains the configurations preferred under declared objectives, risks, tradeoffs, and stakeholder or constitutional criteria. **IC** may be expanded interchangeably as **Instance Constitution** or **Instance Configuration**; it selects and binds one governed slice for a concrete instance. **IR** may be expanded interchangeably as **Instance Realization** or **Implementation Realization**; it makes that IC concrete in source, artifacts, runtime state, operations, and effects. Evidence E evaluates typed claims about the IR. Revision R can change any earlier layer while preserving lineage.

The source manuscript's unqualified `CM` is retained as shorthand for the configuration-manifold envelope, but its definition of "admissible configurations" corresponds specifically to CM1. The added CM2 layer prevents preference, optimization, safety margin, usability, cost, sovereignty, or institutional desirability from being mistaken for mandatory admissibility. The source symbol

`I` is normalized here to the IR stage. Neither accepted expansion of IC or IR changes standing or effect.

The extension carries its own rollback condition: if a separation fails to yield diagnostic, governance, selection, or correspondence distinctions unavailable from an undivided layer without equivalent added structure, that separation should be treated as presentational rather than substantive.

3.3 Specification

Specification describes typed semantic possibility rather than one deployment. It can define objects, relations, invariants, operation types, transition constraints, evidence requirements, possible authority structures, and forbidden states. Formal methods can give selected portions much greater rigor than the notation used here. Specification validity does not itself establish admissibility under a current constitution, instance selection, implementation correspondence, present authority, observed effect, or evidentiary standing.

3.4 Configuration Manifold 1: Admissible Configurations

For mandatory admissibility predicate A_S:

$$CM_1(S) = \{c \mid A_S(c) = 1\}.$$

CM1 may include provider choices, persistence backends, execution profiles, evidence routes, authority arrangements, subsystem activation, security postures, resource envelopes, and effect-realization strategies. Its elements can be mutually exclusive, dependent, unreachable from the current state, evidence-sensitive, environment-sensitive, authority-gated, or semantically similar while operationally distinct.

Admissibility is typed and scoped. A configuration may satisfy structural or syntactic requirements while remaining undesirable, fragile, expensive, invasive, or unsafe under a richer preference constitution. That is why CM1 must not absorb CM2.

3.5 Configuration Manifold 2: Preferred Admissible Configurations

Let $\preceq_{\Pi,t}$ be a declared, versioned, possibly partial preference relation under preference constitution Π and evaluation context or time t. Then a simple nondominated form is:

$$CM_2(S, \Pi, t) = \{c \in CM_1(S) \mid \nexists c' \in CM_1(S) : c \prec_{\Pi,t} c'\}.$$

This is one selection model, not a universal optimizer. Preferences may be incomplete or contested; several configurations may remain incomparable; and safety or rights constraints that are mandatory belong in CM1 rather than as compensable preferences. CM2 can consider least authority, inspectability, privacy, sovereignty, reversibility, performance, cost, maintainability, evidence quality, accessibility, and institutional fit without pretending those objectives collapse into one scalar score.

3.6 Instance Constitution / Instance Configuration

A concrete system does not instantiate either manifold in full. IC binds one selected configuration or constrained path:

$$IC \in CM_2 \subseteq CM_1.$$

The relation expresses intended placement, not proof that selection was legitimate or that an IR corresponds. An IC may bind provider, storage strategy, environment profile, authority topology, enabled subsystems, evidence profile, release lineage, effect-governance profile, realization strategy, and revision policy. A capability may exist in Specification or CM1, even be preferred in CM2, yet remain unselected by the current IC.

3.7 Instance Realization / Implementation Realization

IR is the concrete realization of the selected constitutional pathway: source, binaries, firmware, files, APIs, runtime processes, persistent state, UI behavior, hardware mutations, and external effects. IR cannot retroactively authorize itself. A repository or runtime can contain dormant, stale, unreachable, unselected, or unmodeled behavior. Those observations establish implementation facts only within their evidence boundary.

3.8 Evidence and Revision

Evidence binds observations, subjects, methods, contexts, and claim scopes. It may establish correspondence, defect, noncorrespondence, refusal, uncertainty, or absence of sufficient support; it does not automatically grant authority or standing. Revision can repair Specification, either manifold, the IC, the IR, the evidence model, or their relations. A successor must preserve what changed, why, under whose authority, and which earlier evidence was invalidated.

3.9 Diagnostic Value of the Expanded Split

For one unexpected observation, the expanded representation separates at least these hypotheses:

```
Is the required possibility missing or not yet specified?
Is the candidate absent from CM1 because it violates mandatory
admissibility?
Is it in CM1 but excluded or dominated under the declared CM2
preferences?
Is it preferred but not selected by this IC?
Was it selected but not realized by this IR?
Was it realized but inadequately observed or evidenced?
Does new evidence require revision of an earlier layer?
```

The split earns its cost only when these distinctions change diagnosis, permissible action, or the evidence required for promotion.

4. Dynatology, Co-Projection, and the Definition of Reasoning Capability

4.1 Dynatology as possibility semantics

The Configuration Manifold connects directly to **Dynatology**: the study of possible continuation.

Given current state (x), a system may distinguish:

- conceivable continuations;
- specification-valid continuations;
- admissible continuations;
- reachable continuations;
- viable continuations;
- selected continuations;
- authorized continuations;
- realized continuations;
- observed continuations;
- established continuations.

A simplified representation is:

$$D_x = (x, \Omega_x, A_x, R_x),$$

where:

- Ω_x is the conceivable continuation space;
- A_x describes admissibility;
- R_x describes reachability.

The progression is:

```
conceivable

specification-valid

admissible

selected

authorized

realizable

realized
```

```
observed

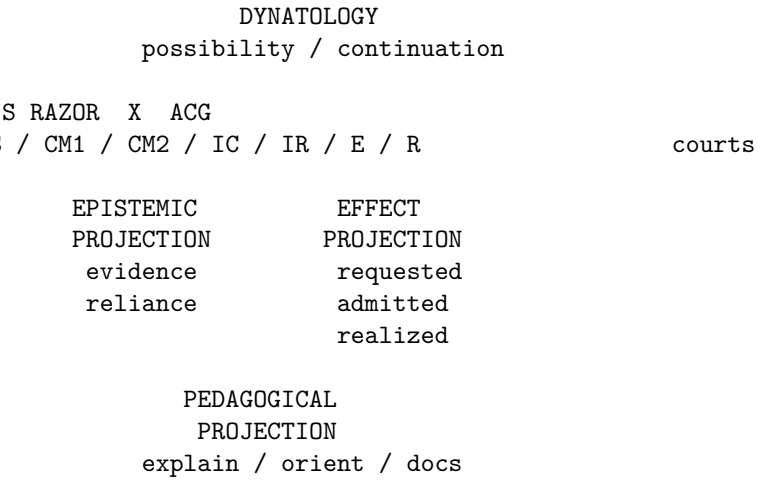
established
```

These are related but non-identical categories.

ACG can therefore be understood partly as a **governed dynatological traversal architecture**.

4.2 The projection nexus

A governed phenomenon (X) can be represented simultaneously through several projections:



No single projection is required to contain the entire truth.

Their intersections can nevertheless reveal distinctions unavailable in isolation.

Consider:

```
Dynatology:
  behavior C is conceivable.

Specification:
  C is valid.

Configuration Manifold:
  C is available.

Instance Constitution:
  C was not selected.
```

```
Authority:
  no transition authorized selection of C.

Implementation:
  C occurred.

Effect evidence:
  C was observed.
```

The combined conclusion is:

Unselected or unauthorized realization residue exists.

The underlying runtime observation did not change.

The representation changed.

4.3 Operational definition of reasoning-capability gain

The phrase **reasoning capability** is easily overstated.

This paper therefore uses a narrower operational definition.

Let (X) be a fixed observed system state, (R_1(X)) a baseline representation, and (R_2(X)) an enriched representation.

The enriched representation provides a reasoning-capability gain when it enables derivation of a decision-relevant distinction, contradiction, obligation, or restriction on admissible action that cannot be expressed or derived under the baseline representation without importing equivalent additional structure.

Formally, for some decision-relevant proposition (q):

$$q \notin Derive(R_1(X))$$

while:

$$q \in Derive(R_2(X)),$$

subject to the condition that (R_2) is not merely hiding the same distinction under a renamed equivalent representation.

A useful informal summary is:

Reasoning Gain = New Discriminations + New Derivable Constraints + New Action Distinctions

while the underlying observation remains fixed.

This definition avoids claiming that the system has acquired general intelligence merely because more metadata exists.

The gain must affect what can be **distinguished or derived**.

For example:

Baseline configuration representation

```
behavior C exists
configuration supports C
```

CM1/CM2/IC representation

```
C exists in admissible configuration space
BUT
this instance never selected C
```

The second representation supports the additional diagnosis:

```
UNSELECTED_REALIZATION_RESIDUE
```

Similarly:

Baseline effect interception

```
request was modified
```

Constitutional effect representation

```
requested operation
\neq
admitted realization
\neq
actual effect
```

This enables detection of executor-correspondence defects that the first representation cannot express without importing equivalent distinctions.

The proposed capability upgrade is therefore **architectural and representational**, not a claim that the underlying language model has become intrinsically more intelligent.

---

5. Deterministic Projection and Constitutional Documentation

5.1 Documentation should not become a second ontology

Documentation is commonly maintained beside the system it describes.

A README, glossary, architecture document, configuration reference, and source implementation can all disagree.

Under constitutional synchronization, these surfaces should instead be treated where possible as **deterministic projections**.

For governed semantic state:

$$X = (S, CM_1, CM_2, IC, IR, E, R, F),$$

and projection function (P_k):

$$P_k(X) = D_k.$$

Possible projections include:

- README;
- glossary;
- implementation codex;
- configuration reference;
- capability map;
- current-instance orientation;
- operator help;
- architecture views.

For a fixed source state and deterministic projector:

$$P_k(X)_1 = P_k(X)_2.$$

This yields a useful principle:

Documentation becomes disposable but reproducible.

"Disposable" does not mean unimportant.

It means the prose file is not the authoritative source of the semantics it explains.

Delete the projection and regenerate it.

The underlying semantic state survives.

5.2 Typed projections

Different surfaces can expose different regions of the nexus.

Specification projection

What is this concept?

Configuration-Manifold projection

What configurations and realization alternatives are admissible?

Instance-Constitution projection

What has this system actually selected?

Implementation projection

Where and how is that selection realized?

Epistemic projection

What is currently established, defeated, defeasible, or unknown?

Effect projection

What was proposed, admitted, executed, and observed?

Pedagogical projection

How should a human or fresh agent understand the current system?

This is more precise than attempting to make one README explain every representational layer.

5.3 Projection does not establish correspondence

Generated documentation alone does not solve specification drift.

A projection can perfectly reproduce a false model.

Therefore:

$$\text{projection fidelity} \neq \text{realization fidelity}.$$

The full loop must include:

`MODEL`

`deterministic projection`

`MODEL`

`realization obligations`

`IMPLEMENTATION`

`witness`

`EVIDENCE`

`COURTS`

`CORRESPONDENCE`

Documentation becomes trustworthy only within the limits of the correspondence that has actually been established.

5.4 Projection is zero-authority

A documentation surface must not become law merely because it is generated.

Thus:

$$\text{projection} \neq \text{law}$$

and:

$$\text{description} \neq \text{standing}.$$

Deleting documentation should not alter execution semantics.

Editing a help page should not amend the Instance Constitution.

A projection explains the governed source; it does not ratify it.

5.5 Projection congruence as an executable obligation

Deterministic projection creates another useful court surface.

Let (D_c) be the committed documentation projection and (P_k(X)) be a fresh regeneration from the current source state.

Projection congruence requires:

$$D_c = P_k(X).$$

Failure establishes only that the committed projection does not correspond to the current projection source.

It does **not** automatically establish which side is substantively correct.

That distinction matters.

A stale README and a stale semantic source can both produce projection mismatch.

The mismatch generates an obligation to adjudicate the divergence; it does not decide the answer by itself.

This principle applies equally to the manuscript describing the architecture. A mathematically intended expression that is silently transformed during transport into a Markdown heading is not merely a typographic defect. It is a projection-correspondence defect between the authored semantic representation and the realized document artifact.

---

6. Constitutional Effect Interposition

6.1 From admission to realization selection

Governance is often conceived as a binary gate:

$$E \to \text{ALLOW}, \text{DENY}.$$

That is useful but incomplete.

A governed system can instead treat the application's requested operation as a proposal whose concrete realization remains selectable.

Let:

- (O_r) = requested semantic operation;
- (R_a) = admitted realization;
- (E_a) = actual effect.

Then:

$$\boxed{O_r \to R_a \to E_a}$$

and the admitted realization need not be identical to the realization originally requested by the application:

$$R_a \neq R_r$$

may be lawful.

Possible dispositions include:

```
PRESERVE
NEGATE
SUBSTITUTE
TRANSFORM
CONSTRAIN
DEFER
```

```
SIMULATE
SANDBOX
ESCALATE
```

The constitutional layer therefore does more than decide whether a call proceeds.

It can select the lawful realization of the operation.

6.2 The constitutional effect membrane

The pattern is:

```
APPLICATION LOGIC

PROPOSED OPERATION

TYPED EFFECT ADAPTER

CONSTITUTIONAL ADJUDICATION

ADMITTED REALIZATION

EXECUTOR

ACTUAL EFFECT

RECEIPT / WITNESS
```

For example, a semantic operation:

```
SAVE_GAME
```

might have several admissible realizations in the CM:

```
classic filesystem path
isolated namespace
temporary diagnostic namespace
simulated/no-write
refusal
```

CM1 establishes the mandatorily admissible set; CM2 ranks or filters it under declared preferences; the current IC then binds the selected slice.

A classic profile might preserve the legacy path.

An isolated profile might substitute a governed namespace.

A diagnostic profile might select the same write while increasing witness capture.

A read-only profile may negate the write.

The operation remains semantically recognizable.

Its realized effect is constitution-dependent.

6.3 Runtime traversal of the Configuration Manifold

Suppose operation (O) admits realization set:

$$R(O) = r_1, r_2, \ldots, r_n.$$

The IC constrains that set:

$$R_{IC}(O) \subseteq R(O).$$

Current evidence, authority, environment, and jurisdiction may further restrict it. Let those constraints induce admissible subsets (R_A), (R_E), and (R_J). Then:

$$R^*_{IC}(O) = R_{IC}(O) \cap R_A \cap R_E \cap R_J.$$

Execution becomes a local dynatological traversal:

```
operation proposed

lawful realization possibilities

instance selection

current authority/evidence constraints

selected realization or refusal

actual effect
```

This creates a direct bridge among the Configuration Manifold, Instance Constitution, Dynatology, and runtime execution.

6.4 Behavioral overlays over legacy software

The significance for legacy systems is considerable.

Suppose the original behavior graph is:

$$G = (V, E).$$

Instead of rewriting all nodes and control logic, consequential edges (e_i) can be routed through constitutional transformation:

$$e_i \to C(e_i),$$

where:

$$C(e_i) \in \{e_i, \varnothing, e_i', defer(e_i), simulate(e_i), \\ sandbox(e_i)\}.$$

The constitutionally realized graph becomes:

$$G_C.$$

Much of the original program can remain structurally intact.

This creates a **constitutional behavioral overlay**.

The architecture can alter consequential behavior without requiring every upstream domain decision to be rewritten.

6.5 Capability grants as effect choke points

A legacy migration may begin with conditional wrappers around consequential operations, but a mature architecture should avoid scattering arbitrary governance conditionals through domain logic.

A stronger structure is:

```
DOMAIN CALL

TYPED EFFECT ADAPTER

CAPABILITY REQUEST

UGK / GOORRT

RECEIPTED DISPOSITION

EXECUTOR
```

This separates the semantic operation from the mechanism through which authority over its realization is granted.

The effect adapter can expose:

- operation identity;
- target object;
- requested realization;
- relevant IC state;
- available evidence;

- authority presented;
- expected effect class.

The governor can then preserve, substitute, constrain, defer, simulate, or refuse the effect without forcing the original application logic to understand the full governance substrate.

6.6 Proposal capability is not effect capability

This has direct implications for agentic systems.

A generative agent may be allowed broad proposal capability without receiving equivalent power to act.

Thus:

$$\boxed{\text{proposal capability} \neq \text{effect capability}}$$

The agent may propose:

- a file write;
- process launch;
- network request;
- hardware mutation;
- release emission;
- deployment action.

The constitutional layer separately determines whether and how that proposal can be realized.

Exploration can remain broad.

Consequence can remain bounded.

---

7. Receipts, Causal Observability, and Development Evidence

7.1 Effect boundaries become observation boundaries

Once consequential operations cross explicit boundaries, those boundaries become natural witness points.

An effect receipt may record:

```
subject
operation proposed
originating component
target object
Instance Constitution
```

```
relevant configuration
available evidence
authority presented
court / GOORRT finding
constitutional disposition
requested realization
admitted realization
executor result
post-effect witness
correspondence status
```

This changes debugging.

Instead of:

> Something wrote this file.

the system may reconstruct:

```
operation:
  SAVE_GAME

requested realization:
  legacy path

disposition:
  SUBSTITUTE

admitted realization:
  isolated namespace

executor:
  success

post-effect witness:
  subsequent resolver consumed same namespace
```

The governance membrane becomes a causal observatory.

7.2 Realization-chain probes

This is especially important when a configuration is generated successfully but not consumed by runtime behavior.

Suppose:

```
requested:
  readable_serif

generated config:
```

```
  readable_serif

configuration write:
  PASS
```

A conventional workflow may overcompress this to:

```
FONT_ACTIVE = true
```

A constitutionally synchronized realization chain can instead preserve:

```
FONT_CONFIGURATION_REALIZED = true
FONT_RUNTIME_CONSUMPTION = NOT_ESTABLISHED
FONT_RENDERED_EFFECT = NOT_ESTABLISHED
```

The architecture distinguishes:

```
requested
configured
consumed
executed
observed
established
```

This is a practical example of a representational reasoning gain.

7.3 Receipts do not prove themselves

Receipt production must not be confused with correctness.

A receipt can establish that a governed component recorded a disposition.

It does not, by itself, establish that:

- the recorder observed reality correctly;
- the executor obeyed the disposition;
- the target effect occurred;
- the evidence route was independent;
- the subject has standing.

Thus:

$$\text{receipt} \neq \text{proof of correctness.}$$

Receipts become powerful when combined with independent or orthogonal witnesses and correspondence courts.

---

8. Development Cases

The following episodes are reported at explicitly different evidentiary strengths. This distinction is necessary because a paper about subject-bound evidence would undermine itself if all development anecdotes were narrated at one undifferentiated epistemic level.

8.1 Environmental safety versus realization fidelity

A donor archive was extracted using a safer archive-extraction mechanism.

The environment normalized file permissions.

A read-only Grundnorm self-check moved from 5/5 to 4/5.

The relevant constraints were:

$$A = \text{archive extraction must remain structurally safe}$$

$$B = \text{donor realization must correspond}$$

$$C = \text{Grundnorm must remain read-only}$$

$$D = \text{court may not be weakened merely to obtain PASS.}$$

The new environmental fact was:

$$E = \text{safe extraction modified permission semantics.}$$

No bespoke rule had said:

> Restore permissions after safe extraction.

Instead, the composed constraints excluded several apparent fixes.

Ignoring the permission drift violated realization fidelity.

Weakening the self-check violated the court constraint.

Abandoning structural extraction safety weakened the custody constraint.

The remaining solution region was:

```
validate archive path/link safety

establish safe custody
```

```
preserve semantically significant donor metadata

rerun unchanged court

5/5
```

This is an example of compositional constraints narrowing the realization space.

Evidentiary status This episode is author-reported from the development record. It is included for its compositional structure and is not presented here as independently established evidence.

8.2 Reuse is not materialization

A repair path identified a previously established artifact as reusable.

The convenient implementation copied that artifact into the current workspace inside the reuse operator.

Functionally, the correct bytes appeared.

Constitutionally, the operation decomposition was wrong.

The reuse decision meant:

> This receipt-bound identity remains admissible for reuse.

Materialization meant:

> Write concrete bytes into a local path.

Assembly meant:

> Consume those bytes within an authorized integration operation.

The naive implementation produced:

```
REUSE DECISION

hidden write
```

making a nominal skip path effectful.

The corrected decomposition was:

```
REUSE DECISION
  effect = NONE

AUTHORIZED ASSEMBLY

MATERIALIZATION
  verify source receipt
```

```
verify expected digest
treat local path as untrusted cache
rematerialize if missing/tampered
reverify result
```

```
ASSEMBLY RECEIPT
```

A hostile local-copy mutation was then handled by restoring exact receipt-bound bytes rather than trusting remembered path identity.

The resulting non-collapse rules are:

$$\text{evidence reuse} \neq \text{effect reuse}$$

$$\text{identity reuse} \neq \text{path trust}$$

$$\text{skip} \neq \text{write}.$$

Evidentiary status The decomposition is preserved in the governed construction path of the reference implementation. Surrounding verification was externally replayed, while the particular hostile-tampering fixture described here was not independently re-executed by that reviewer. The status is therefore artifact-preserved and partially externally replayed.

8.3 Evidence belongs to the subject

Agentic sessions are vulnerable to the reasoning shortcut:

> We proved that already.

ACG instead binds verification to an exact subject commitment:

$$R_S = H(S)$$

and plan commitment:

$$R_P = H(P).$$

If a source or model change produces:

$$R_{S_1} \neq R_{S_2},$$

previous evidence cannot silently certify the new subject.

In a development episode, a source change invalidated the prior subject root. Earlier leaf evidence was not carried forward merely because it had once passed. The verification DAG was re-executed under the new subject and plan.

The principle is:

> **Evidence belongs to a committed semantic subject, not to the agent's memory that something similar was previously verified.**

Evidentiary status Subject-bound evidence invalidation has been repeatedly exercised in the release lineage and externally replayed. External reviewer provenance can itself be typed separately from proof contents; replay by a distinct host does not automatically establish reviewer independence.

8.4 The verifier that testifies to its own supersession

Repositories accumulate historical verification tooling.

Deleting old tools destroys provenance.

Leaving them looking current creates orientation hazards.

One historical successor gate, when executed on a later generation, initially produced a misleading refusal.

The repair was not deletion.

The historical tool was changed so that on the newer subject it returned:

```
HISTORICAL_SUPERSEDED
pass = true
authority = NONE
standing = NONE
```

and routed the caller toward the current verifier.

The historical mechanism therefore remained reproducible without masquerading as current jurisdiction.

Evidentiary status The problem was first identified outside the system's own immediate review path and then incorporated into the corrected release surface. Among the cases reported here, this provides the strongest example of an externally originated architectural finding.

---

9. Agentic Development as Governed Search

The architecture implies a division of computational responsibility.

9.1 Generative agent

The agent excels at:

- hypothesis formation;
- code synthesis;
- architecture search;
- debugging;
- analogical transfer;
- candidate repair generation.

9.2 Semantic compiler

The semantic layer:

- types the subject;
- compiles obligations;
- exposes model/realization distinctions;
- identifies applicable courts;
- constrains transitions;
- maps operations into admissible realization spaces.

9.3 Evidence mechanisms

Evidence systems:

- run witnesses;
- bind hashes;
- capture observations;
- preserve provenance;
- report uncertainty.

9.4 Courts

Courts:

- adjudicate correspondence;
- preserve typed dispositions;
- reject inadmissible claim migration;
- test effect correspondence;
- identify unresolved obligations.

9.5 Authority and execution

Authority determines which consequential transitions may proceed.

The executor performs only the admitted realization.

The resulting architecture is:

```
AGENT
  proposes / searches

SEMANTIC REPRESENTATION
  structures the problem

DETERMINISTIC CONSTRAINTS
  narrow admissible possibilities

EVIDENCE
  grounds observations

COURTS
  adjudicate

AUTHORITY
  permits / constrains / refuses

EXECUTOR
  effects

RECEIPTS
  preserve causality and lineage
```

The language model remains important.

It is not made the final arbiter merely because it is capable of proposing solutions.

---

10. A New Software-Development Process Model

The conventional project model may be simplified as:

```
requirements
source
tests
build
documentation
artifact
```

The constitutionally synchronized project becomes:

```
SPECIFICATION

CONFIGURATION MANIFOLD 1

CONFIGURATION MANIFOLD 2
```

```
INSTANCE CONSTITUTION / CONFIGURATION

MODELED OPERATIONS

REALIZATION OBLIGATIONS

IMPLEMENTATION

PROPOSED EFFECTS

CONSTITUTIONAL EFFECT INTERPOSITION

ACTUAL EFFECTS

EVIDENCE / RECEIPTS

COURTS

EPISTEMIC / RELIANCE STATE

AUTHORITY / STANDING
```

with deterministic projections available alongside the governing semantic source.

The unit of development is therefore not merely source code.

It is a **constitutionally synchronized semantic and effect state**.

The corresponding process progression is:

CI/CD

```
change
-> build
-> test
-> package
-> deploy
```

ACG

```
proposal
-> model
-> locate admissible possibility
-> select instance configuration
-> realize
-> propose effect
-> adjudicate realization
-> execute
```

```
-> witness
-> establish correspondence
-> receipt
-> establish permitted standing
-> project
```

CI/CD remains inside this architecture.

The enlargement can be described as:

> **Continuous constitutionalization and adjudication of governed change.**

The progression is:

```
tests become infrastructure

courts become infrastructure

correspondence becomes infrastructure

effect governance becomes infrastructure

causal observability becomes infrastructure

documentation becomes deterministic projection
```

---

11. Related Work and Ancestry

The following comparisons are intended to narrow the proposed contribution rather than inflate novelty. Surface resemblance is not validation; neighboring traditions often solve portions of the same problem with greater maturity or formal rigor.

11.1 Continuous integration and delivery

Resemblance. Both make verification an automatic consequence of change rather than an optional activity (Fowler, 2006; Duvall, Matyas, and Glover, 2007; Humble and Farley, 2010).

Distinction. Conventional CI/CD principally orchestrates predicates over builds and artifacts. ACG adds a representational layer in which selected verifications are bound to subjects, frames, evidence classes, jurisdictions, and authority consequences.

Misuse prevented. Treating a green pipeline as a complete epistemic position.

11.2 Model-driven engineering

Resemblance. Model-driven engineering has long treated models as central engineering artifacts and has explored generation, refinement, synchronization, and bidirectional relationships among representations.

Distinction. The relevant divergence is not that MDE lacks synchronization. Many MDE traditions address synchronization explicitly. The narrower ACG claim is that neither model primacy nor correctness of a transformation is itself sufficient to establish constitutional authority, evidentiary standing, or present realization correspondence. A model may guide or generate implementation, but claims about the current realized subject remain separately adjudicable.

Misuse prevented. Treating correct projection from a model as proof that the current runtime corresponds, or treating model primacy as constitutional authority.

11.3 Software product lines and variability modeling

Resemblance. Feature and variability models describe admissible configuration spaces with dependencies, exclusions, and alternative selections. Dynamic software product lines also support runtime configuration and reconfiguration.

Distinction. Runtime variability is therefore not the distinguishing contribution. The ACG-specific addition is treating the concrete Instance Constitution as a provenance-bearing, authority-sensitive commitment whose transitions may themselves require evidence, jurisdiction, receipts, and adjudication. Existence of a variant in configuration space does not imply selection by the current instance.

Misuse prevented. Confusing "available configuration" with "selected constitutional state."

11.4 Policy as code and admission control

Resemblance. Policy engines and admission controllers externalize executable rules evaluated around consequential requests. Some admission systems support mutation rather than simple binary acceptance.

Distinction. Mutation itself is therefore not novel. The ACG-specific structure is the explicit separation and later correspondence checking of:

$$\begin{aligned} &\text{requested operation} \\ &\rightarrow \text{admitted realization} \\ &\rightarrow \text{actual effect} \\ &\rightarrow \text{evidence} \end{aligned}.$$

The architecture asks not only how a request should be changed before admission, but what realization was constitutionally selected, what the executor subsequently did, and what evidence establishes correspondence between the two.

Misuse prevented. Treating request mutation as equivalent to effect correspondence.

11.5 Reference monitors, capabilities, effect handlers, and interposition

Resemblance. Reference monitors, capability systems, security kernels, system-call mediation, aspect-oriented programming, and algebraic effect handlers all demonstrate the power of interposing behavior at consequential boundaries (Saltzer and Schroeder, 1975).

Distinction. These traditions are not authority-neutral; many are explicitly about authority. The narrower difference is that they generally do not combine mediation with the complete ACG tuple of exact-subject binding, constitutional frame, evidence admissibility, claim jurisdiction, correspondence receipts, lineage, and standing. Conversely, execution monitors can enforce only certain classes of properties, a limitation inherited by any constitutional effect membrane (Schneider, 2000).

Misuse prevented. Treating interception alone as constitutional governance or assuming an effect boundary can enforce arbitrary semantic properties.

11.6 Formal methods

Resemblance. Formal specifications, model checkers, and SMT solvers constrain admissible states and prove properties with rigor far beyond the informal equations used in this paper (Lamport, 2002; Clarke et al., 2018; de Moura and Bjrner, 2008). Production use demonstrates that these methods are not restricted to toy systems (Newcombe et al., 2015).

Distinction. A verified model does not necessarily establish that a currently deployed implementation corresponds to that model, that the evidence belongs to the exact current subject, or that a successful proof confers authority for a consequential transition. ACG treats formal proof as potentially powerful evidence whose admissibility and jurisdiction remain typed.

Misuse prevented. Treating formalization itself as a promotion of claim standing.

11.7 Testing theory and the oracle problem

Testing cannot solve the absence of a reliable oracle merely by increasing test volume (Weyuker, 1982).

Courts do not solve this either.

They type the limitation.

Where a proposition cannot be established, `NOT_ESTABLISHED` is a legitimate disposition rather than forced PASS or FAIL.

Misuse prevented. Treating absence of a detected failure as evidence of correctness.

11.8 Systems safety

Systems safety treats accidents as failures of control across interacting sociotechnical structures rather than isolated component defects (Leveson, 2011).

The effect membrane resembles a control boundary in that sense.

ACG is not itself a safety case, however. Its primary concern is epistemic, constitutional, and authority-preserving development control.

Misuse prevented. Treating governed effects as sufficient hazard analysis.

---

12. Limitations

12.1 Model error

A deterministic architecture can enforce a bad model perfectly.

Constitutional rigor cannot substitute for criticism of the Specification itself.

12.2 Representation multiplication

Additional projections can produce reasoning gains but also complexity.

Each projection should earn its existence by generating distinctions that matter.

The CM1/CM2/IC split is explicitly subject to this requirement.

12.3 Court proliferation

Not every test should become a court.

Constitutional typing should be proportional to semantic and governance significance.

12.4 Effect-interposition proliferation

Not every function call should become a governed effect.

The architecture should identify consequential semantic boundaries rather than constitutionalize ordinary computation.

12.5 False formal confidence

Hashes establish identity, not truth.

Receipts establish recorded transitions, not necessarily correctness.

Courts establish only the propositions within their actual jurisdiction and evidence topology.

Rendered mathematical notation likewise does not make an argument valid, but malformed rendering can destroy the intended semantics of a valid argument. Representation integrity and argument validity are distinct obligations.

12.6 Evidence quality

Software often offers unusually exact evidence:

- byte identity;
- deterministic replay;
- precise hashes;
- reproducible execution.

Other domains may rely on incomplete, probabilistic, contested, or human evidence.

The architecture must preserve uncertainty rather than fabricate precision.

12.7 Optimization against courts

Agents may learn to satisfy visible predicates without satisfying intended semantics.

Hostile cases, anti-vacuity checks, correspondence witnesses, and independent evidence remain necessary.

12.8 Governor correctness

An effect governor is itself a high-leverage implementation.

Errors in operation typing, capability mapping, realization selection, or executor binding can cause consequential failures.

The governance layer therefore requires the same constitutional synchronization it imposes elsewhere.

12.9 Self-ratification

A governance system cannot simply declare its own legitimacy.

A court running inside the subject, a projection explaining the subject, or a governor receipting its own decision may contribute evidence, but subject-controlled evidence cannot automatically become sufficient for protected standing.

This paper depends on that constraint but does not attempt to solve the full reflexive-legitimacy problem.

---

13. Research Program

The architecture produces several testable hypotheses.

H1 – Courts reduce unsupported completion claims

Agents operating with constitutionally typed verification will make fewer unsupported "done" claims than agents operating with conventional test suites alone.

H2 – CM1/CM2/IC improves defect localization Agents given P/S/CM1/CM2/IC/IR/E/R representations will more accurately distinguish:

- missing possibility;
- malformed configuration space;
- wrong instance selection;
- implementation defect.

H3 – Co-projection produces new actionable distinctions Holding observed system state constant, a richer projection nexus will enable derivation of contradictions or repair classifications unavailable under a baseline representation without equivalent added structure.

H4 – Deterministic projection reduces documentation drift Generated source-bound documentation will disagree less often with verified semantic state than manually maintained documentation.

H5 – Subject-bound evidence reduces stale carry-forward Content-addressed subject and plan commitments will reduce reuse of verification conclusions across materially changed implementations.

H6 – Governed constraint improves agent search Agents operating inside deterministic constitutional constraints will converge on solutions satisfying more independent obligations than equivalent agents receiving conventional tests alone.

H7 – Effect interposition reduces hidden consequential behavior Systems in which consequential effects cross typed governance boundaries will exhibit fewer unmodeled writes, launches, persistent mutations, and capability uses.

H8 – Causal receipts improve fault localization Receipt chains will reduce ambiguity in distinguishing requested, configured, admitted, executed, observed, and established behavior.

H9 – Behavioral overlays reduce legacy-fork divergence Legacy systems adapted through governed effect boundaries will preserve more upstream structure than equivalent systems modified through deep domain-logic rewrites.

H10 – Proposal/effect separation increases safe autonomy Agents will be able to retain broader proposal capability without proportional increases in uncontrolled consequential authority.

---

14. Experimental Design

A useful ablation study would compare four agent conditions over equivalent repository tasks.

Group A – Conventional Agent + CI

Receives:

- repository;
- task;
- conventional test suite.

Group B – Agent + Courts

Additionally receives:

- court bindings;
- typed dispositions;
- subject/frame association.

Group C – Agent + Constitutional Projection Nexus

Additionally receives:

- P/S/CM1/CM2/IC/IR/E/R representation;
- evidence topology;
- deterministic orientation;
- subject-bound verification plans.

Group D – Projection Nexus + Governed Effects

Additionally operates through:

- typed effect adapters;
- constitutional realization selection;
- receipt-before-effect governance.

Metrics can include:

- unsupported completion claims;
- repair classification accuracy;
- stale evidence reuse;
- hidden effects;
- court weakening attempts;
- unauthorized effects;
- requested-versus-actual-effect mismatch;
- iterations to valid realization;
- documentation drift;
- reviewer cold-boot reconstruction accuracy;
- novel contradictions discovered before human review;
- fault-localization time from receipt chains.

The critical experimental question is not simply whether Group D performs better.

It is **which representational addition accounts for which capability gain**.

That is the purpose of the ablation ladder:

$$\begin{aligned} &\text{Tests} \\ &\rightarrow \text{Courts} \\ &\rightarrow \text{Projection Nexus} \\ &\rightarrow \text{Governed Effects} \end{aligned}.$$

The design also permits a stronger test of the reasoning-capability definition from Section 4.3. For a fixed observable defect (X), one can ask which experimental groups can derive a required distinction (q), and whether that distinction changes the repair selected by the agent.

---

15. Implications Beyond Software

The paper's principal claims concern agentic software development.

The underlying representation is more general.

The sequence:

$$
\begin{aligned}
& P \\
& \to S \\
& \to CM_1 \\
& \to CM_2 \\
& \to IC \\
& \to IR \\
& \to E \\
& \to R
\end{aligned}
$$

and the distinctions among:

- evidence;
- admissibility;
- authority;
- effect;
- standing;

can be projected into other domains.

15.1 Construction

```
Specification
  engineering and code constraints

CM
  admissible project realizations

IC
  selected project/site constitution

Operation
  install / modify / inspect

Implementation
  actual physical work

Evidence
  measurements / photographs / inspections

Courts
  code / safety / scope / correspondence

Authority
  owner / engineer / inspector / foreman
```

15.2 Organizational governance

```
Specification
  charter / policy / budget rules

CM
  admissible organizational options

IC
  selected operating plan

Operation
  hire / spend / contract

Implementation
  actual commitment

Evidence
  records / financial observations

Courts
  compliance / authority / risk
```

15.3 Clinical workflows

A similar non-collapse is immediately useful:

$$\begin{aligned}&\text{observation}\\ &\neq \text{diagnosis}\\ &\neq \text{treatment selection}\\ &\neq \text{authorization}\\ &\neq \text{execution}\\ &\neq \text{outcome}\end{aligned}.$$

15.4 Legal systems

Many of the terms are already native:

- jurisdiction;
- evidence;
- admissibility;
- authority;
- disposition;
- standing;
- execution.

Software is attractive as the initial proving ground because its evidence can be unusually exact.

Generalization to less deterministic domains requires more–not less–care around uncertainty.

---

16. Discussion

The transition from tests to courts can initially appear to be a testing innovation.

It is better understood as one component of a larger change in the software-development process model.

Five transformations interact:

CI/CD	: verification becomes infrastructure
ACG Courts	: adjudication becomes infrastructure
Constitutional Synchronization	: correspondence becomes infrastructure
Effect Interposition	: consequence becomes governed infrastructure
Deterministic Projection	: explanation becomes reproducible infrastructure

Ableman's Razor supplies a typed representation boundary.

The CM1/CM2/IC extension distinguishes available configuration from selected instance commitment.

Dynatology describes the structure of possible continuation.

Evidence grounds observations.

Courts adjudicate claims and correspondence.

Authority limits what may proceed.

Effect boundaries separate proposed operations from realized consequences.

Receipts preserve causal history.

Deterministic projections make the same governed model legible to humans and agents.

The result is not that the generative agent becomes omniscient.

The result is that the agent operates inside a world with more machine-addressable distinctions.

The constitutional substrate can tell an agent:

- the proposed fix violates another invariant;

- the required solution does not exist in the current Specification;
- the solution exists in the CM but was not selected by the IC;
- the implementation does not correspond to the selected configuration;
- a skip path contains a hidden effect;
- old evidence belongs to a previous subject;
- a generated configuration is not observed at runtime;
- a historical verifier lacks current jurisdiction;
- an operation is admissible only through a substituted realization;
- the executor performed an effect different from the admitted realization;
- the documentation projection is current even though realization correspondence is not established.

These are not merely more facts.

They are **additional relations among facts**.

That is the central reasoning claim.

The effect boundary sharpens the claim further. The same domain-level request may have multiple possible realizations. Constitutional reasoning can therefore operate not only over whether an action may occur but over **which admissible manifestation of the operation may become real**.

That is a dynatological question, a configuration question, an authority question, and an implementation question simultaneously.

The projection nexus makes those views composable.

Constitutional Development and Assurance Object Model

The completed P1 corpus gives ACG a bounded role: it is a **constitutional software-development process and assurance model** for agent-generated or agent-modified systems. It joins specification, configuration, adjudication, effect mediation, evidence, and revision without claiming that courts replace the larger testing, delivery, security, operations, or human-review estate.

- **Adjacent work:** *Governance as Compilation*
  Supplies: Governed translation, lowering, semantic remainder, and staged effect.
  ACG boundary: ACG applies those ideas to a development lifecycle and court-mediated assurance.
- **Adjacent work:** *Reasoning by Way of Receipts*
  Supplies: Records of reasoning, transitions, refusals, and observations.
  ACG boundary: A receipt is an evidence carrier; it is not the court, verdict, effect, or standing.
- **Adjacent work:** *Legitimacy Without Self-Ratification*
  Supplies: Independence limits on the strongest assurance claims.

ACG boundary: An agent, generator, or governor cannot establish its own highest standing by self-test.

- **Adjacent work:** *What Software Systems Actually Receipt, Govern, and Hide*
  Supplies: Analysis of visibility and governance gaps in ordinary software.
  ACG boundary: ACG supplies a candidate process response, not proof that every gap is closed.
- **Adjacent work:** *Universal Governance Kernel (UGK)*
  Supplies: A proposed reusable governance-kernel architecture.
  ACG boundary: UGK may realize process components; ACG is the development and assurance model, not one implementation.

The portable development case should bind:

$$D = (proposal, subject, specification, CM1, CM2, IC, IR, \\ courts, authority, effect_path, evidence, standing, \\ revision)$$

Here IC means Instance Constitution or Instance Configuration and realization-stage IR means Instance Realization or Implementation Realization. These stages remain distinct from Intermediate Representation uses such as Action IR or Workflow IR.

The process can be summarized as
`propose -> compile -> adjudicate -> admit -`

`> realize -> observe -> revise`, but each transition has a separate subject and outcome. A **test** generates an observation under a fixture. A **verifier** evaluates a declared property. A **court** applies a named adjudication profile to bound evidence. A **gate** mediates progression. A **governor or effect membrane** constrains an effect path. A **receipt** records an event or decision. A **standing decision** states what the assembled evidence currently warrants. None can silently substitute for another.

Three terminal planes must remain separately reportable: the court's verdict, the realized effect state, and the claim or publication standing. `PASS` does not prove that an effect occurred; a successful effect does not prove it was admissible; and either result may remain insufficient for a broad public claim. Unknown and partially observed effects require reconciliation rather than optimistic completion.

The universal GSM can type `PROPOSES_CHANGE`, `COMPILED_UNDER`, `ADJUDICATED_BY`, `OBSERVED_BY_TEST`, `VERIFIED_BY`, `GATED_BY`, `ADMITTED_UNDER`, `REALIZED_AS`, `EFFECT_MEDIATED_BY`, `RECEIPTED_BY`, `HAS_ASSURANCE_STANDING`, and `REVISED_BY`. These relations support comparison with CI/CD while preserving rather than erasing the ordinary delivery estate.

This integration establishes four editorial commitments: adopt constitutional development and assurance model as ACG's official corpus role; adopt the bound development-case object; enforce the test/verifier/court/gate/governor/receipt/standing non-collapse rule; and require separate reporting of verdict, effect, and public-claim standing.

Adversarial Assurance Lifecycle and Custodied Remainder

The fixed-seed comparison with the Governed Knowledge Loop, Work Custody, Semantic Fragmentation, and Constitutional Epistemology under adversarial pressure gives an ACG change case a wider assurance envelope:

$$\begin{aligned} D^* = (&base, proposal, subject, custody, specification, CM1, \\ &CM2, IC, IR, fragmentation_map, threat_model, tests,. \\ &courts, effects, evidence, standing, remainder, revision) \end{aligned}$$

GKL preserves the semantic lifecycle of the proposal and its successor. Work Custody assigns the next move for design, review, counterexample response, effect reconciliation, and rollback without conferring authority. Open obligations remain custodied after a court pass or successful build.

Semantic Fragmentation reconstructs the operative constitution across configuration, fallback, authority, routing, resource, effect, evidence, and projection surfaces. The ACG assurance subject must include the discovered operative paths; adjudicating only the declared model can certify the wrong constitution.

Adversarial semantic pressure tests formally compliant artifacts, receipts, refusals, witnesses, and projections for omission, drift, capture, replay, or strategic framing. The strongest ACG standing therefore depends on independent-enough observation and explicit residual risk, not simply more courts or internally consistent evidence.

The universal GSM can type
`CUSTODIES_ASSURANCE_REMAINDER`,
`RECONSTRUCTS_OPERATIVE_ASSURANCE_SUBJECT`, `RED_TEAMS_CONSTITUTIONAL_ARTIFACT`,
`DETECTS_FORMAL_COMPLIANCE_ATTACK`, `RECONCILES_COURT_AND_EFFECT`,
`WITHHOLDS_PUBLIC_STANDING`, and
`REVISES_AFTER_ADVERSARIAL_FAILURE`.

This randomized review establishes four editorial commitments: adopt the expanded assurance envelope; retain custody for unresolved post-verdict work; require operative-constitution reconstruction; and integrate adversarial semantic-pressure testing with residual-risk reporting.

17. Corpus Integration and Counterevidence

17.1 GMB and Universal GSM Placement

The **Governed Meaning Base (GMB)** is the current name in the author-confirmed `CCF -> CMB -> GMB` lineage. GMB can retain specifications, CM1 and CM2 members, preference constitutions, IC selections, IR bindings, observations, court definitions, evidence, receipts, effects, projections, standing, and revision lineage. It is not itself a court, governor, executor, authority source, or proof that the retained model corresponds to reality.

The universal GSM should distinguish at least: change proposal, exact subject, specification, admissibility predicate, CM1 configuration, preference criterion, CM2 relation, IC selection, IR artifact or process, modeled operation, requested realization, admitted realization, authority grant, effect, observation, evidence item, verification, court, proposition, jurisdiction, disposition, receipt, standing claim, projection, lineage edge, counterevidence, and revision. High-value relations include `SPECIFIES`,
`ADMITS`, `PREFERS_UNDER`, `SELECTS`, `REALIZES`, `PROPOSES`, `AUTHORIZED_BY`, `ADJUDICATED_BY`, `EXECUTED_AS`, `OBSERVED_AS`, `EVIDENCED_BY`, `PROJECTS`, `COUNTEREVIDENCED_BY`, `INVALIDATES`, and `SUCCEEDS`.

17.2 Direct Corpus Interfaces

- *Ableman's Razor* supplies the original claim-migration grammar; the expanded funnel is a later representational extension and does not silently rewrite that paper.
- *Dynatology* supplies a general continuation-space vocabulary. ACG contributes one software-development governance profile rather than establishing a universal dynatological implementation.
- *Admissibility Engineering* supplies constraint, refusal, and evidence discipline for CM1. CM2 adds an explicitly preference-sensitive selection layer.
- *Reasoning by Way of Receipts* explains how receipted transitions can support reasoning without making receipts self-proving.
- *The Three Hash Rule* separates state, admissibility, and meaning commitments; ACG courts must not collapse those questions into a green pipeline result.
- *The Mechanics of Constitutional Enforcement* supplies the topology from declaration through effect; ACG applies that concern to the software-development lifecycle.
- *The Constitutional Observability Platform* develops the observation and projection surfaces required to compare requested, admitted, executed, and observed effects.
- *Model-First Legacy Migration* supplies the overlay and correspondence context for adapting legacy systems while preserving upstream structure.
- *Semantic Fragmentation and Constitutional Structure* identifies the risk

that law, implementation, evidence, and explanation disagree across surfaces.
- GOORRT provides an adjacent object-operation-realization grammar. It should be connected by typed mappings, not merged by shared realization vocabulary.
- *Sovereign Software* supplies the inspectability, reconstruction, substitution, and provider-exit objectives that can inform CM2 without weakening CM1.

These relations are comparison and dependency candidates, not author-approved identity mergers or mutual validation.

A Profiled Constitutional Assurance Lifecycle

Fifteen retained downstream uses specialize ACG for quantitative systems, adversarial alternation, work custody, agent interfaces, communal systems, adversarial epistemology, open-source development, anti-self-ratification, enforcement, receipts, semantic fragmentation, Implementation Codices, knowledge mutation, and the Ball view. They confirm that ACG is a profiled development and assurance lifecycle. It does not replace ordinary CI/CD, every verifier with a court, semantic governance with software testing, or an effect authority with a verdict.

The assurance case is:

$$\begin{aligned} A^* = (&change, subject_generation, constitution, claim_set,\\ &profile, ordinary_tests, courts, oracles,\\ &attack_surface, roles, effect_path, evidence, verdict,\\ &promotion, rollback, remainder, revision) \end{aligned}.$$

A profile must state the consequence class, constitutional questions, ordinary-test estate, court jurisdiction, defect and counterexample identity rules, oracle and trusted base, role independence, evidence topology, effect boundary, promotion authority, rollback or compensation route, and stopping criterion. Courts are warranted only where typed constitutional adjudication changes a decision that ordinary tests cannot adequately express. A clean court result means no retained counterexample was established within the declared surface; it does not prove completeness, truth, safety, independence, or permission to deploy.

Verdict, promotion, execution, publication, and canonical standing remain different transitions. Each requires its own authority and receipt. The same actor or tool may occupy multiple roles only when the profile declares the collapse, evaluates the common-mode risk, and narrows the claim. Knowledge-base changes, public-space changes, and quantitative-model changes can reuse the lifecycle only through domain-specific profiles, competent reviewers, appropriate oracles, and explicit non-transfer of software-court standing.

The universal GSM can type
`INSTANTIATES_ACG_PROFILE`,
`BINDS_COURT_TO_SUBJECT_GENERATION`, `RETAINS_ORDINARY_TEST_ESTATE`,
`DECLARES_COURT_JURISDICTION`, `SEPARATES_VERDICT_FROM_PROMOTION`,
`CHALLENGES_COMMON_MODE_ORACLE`, `PRESERVES_COUNTEREXAMPLE`, and
`DOES_NOT_SELF_RATIFY`.

This reverse-dependency review establishes four editorial commitments: adopt the profiled assurance case; require consequence, oracle, role, evidence, effect, and stopping fields; preserve the ordinary-test estate and bounded court subset; and prohibit verdicts from silently becoming promotion, execution, publication, or canonical decisions.

17.3 Counterevidence and Attack Surface

ACG is weakened, narrowed, or falsified where: (1) court typing adds no decision-relevant distinction beyond ordinary tests plus metadata; (2) the exact subject or constitutional frame can be spoofed; (3) test and court share the same defective oracle; (4) the court's evidence topology omits the consequential effect; (5) `NOT_ESTABLISHED` is optimized into de facto approval; (6) an executor bypasses the effect membrane; (7) a governor is not tamper-resistant, completely mediated, or independently observable enough for the claim made; (8) CM1 omits a mandatory constraint; (9) CM2 launders a rights or safety requirement into a compensable preference; (10) IC selection is stale, ambiguous, or not consumed at runtime; (11) IR behavior depends on hidden environment state; (12) receipts are authentic but incomplete or collusive; (13) deterministic projections faithfully reproduce a false model; (14) documentation generation hides uncertainty or contested meaning; (15) old evidence is carried to a changed subject; (16) concurrency, retry, partial failure, or compensation breaks the requested-admitted-actual chain; (17) court proliferation or effect interposition makes the system unusable; (18) agents learn the visible court surface while violating its purpose; (19) richer representations increase apparent derivability only by importing the answer or an equivalent oracle; or (20) the reported development cases cannot be reproduced from bound artifacts.

The reasoning-capability hypothesis therefore requires a declared inference system, fixed observation and task, matched information access, an explicit baseline, correctness checks for the new conclusion, and accounting for added representation, construction, review, and runtime cost. Merely making proposition q syntactically expressible does not show that the system derives it correctly or usefully.

Courts Are a Constitutional Subset of Verification

Public orientation

ACG extends CI/CD by giving a constitutionally significant subset of verification explicit jurisdiction, authority, evidence, refusal, and effect consequences. Tests

remain broader than courts. A court is not better merely because it is named constitutional; its law, oracle, subject, evidence, and consequences must be inspectable.

Adversarial tests

1. **Court-proliferation attack.** Every test becomes a court, increasing cost and obscuring constitutional significance. Admission requires a reasoned jurisdiction and consequence profile.
2. **Oracle-capture attack.** The court encodes the same mistaken assumptions as the builder. Independent challenges, alternate oracles, and retained counterexamples are required where stakes justify them.
3. **Constitutional-drift attack.** Law, projection, code, tests, and deployment diverge while each remains internally valid. Co-projection needs cross-artifact correspondence and successor receipts.
4. **Self-ratification attack.** A generator or governor changes the law that authorizes itself. Non-delegable amendment authority and external admission gates must remain outside the candidate's claim.
5. **Effect-bypass attack.** A passing court is irrelevant because another route reaches the effect. Courts must bind to actual capability and effect boundaries.
6. **Optimization-against-courts attack.** Agents learn the evaluator rather than the intended obligation. Hidden assumptions, holdouts, mutation, and adversarial alternation expose specification debt.
7. **False-formal-confidence attack.** Typed artifacts and equations are treated as proof despite model error and unaudited remainder. Formality narrows claims; it does not erase premises.
8. **Legacy-overlay attack.** A behavioral overlay claims control while legacy paths, state, or privileges remain outside it. The overlay must declare coverage and bypass tests.

Claim and evidence ceiling

The paper defines ACG, the expanded realization funnel, a projection taxonomy, and ten testable hypotheses. Those hypotheses are a research agenda, not reported results. The development cases illustrate distinctions; they do not establish general productivity, safety, defect reduction, or reasoning-capability gain.

Claims about CI/CD history, related fields, comparative outcomes, or applications beyond software require resolvable external sources and domain-specific review. Clinical, legal, construction, and organizational examples are portability questions, not validated deployments. No external research was added in this pass.

Terminology boundary

IC may mean Instance Constitution or Instance Configuration. In the realization funnel, IR may mean Instance Realization or Implementation Realization. Those interchangeable stage names remain distinct from Intermediate Representation in MCIR and other named representations.

Reader-facing consequence

The public shift is from agents that generate code and hope tests are enough to agents that work inside a declared possibility space, preserve obligations across representations, encounter typed refusal before consequential effects, and leave evidence that can be contested.

18. Conclusion

Continuous integration transformed software engineering by making tests part of the infrastructure of change.

Ableman Constitutional Governance extends that transformation without replacing it.

Within software, courts are a constitutionally typed subset of executable verification:

$$C_{\text{software}} \subseteq V_{\text{executable}}.$$

They bind selected predicates to an exact subject, constitutional frame, evidence topology, jurisdiction, authority relationship, and typed disposition.

But courts alone are not the full architecture.

Constitutional synchronization addresses the longer-standing separation among specification, configuration, implementation, verification, and documentation.

The central representational sequence developed here is:

$$\boxed{P \to S \to CM_1 \to CM_2 \to IC \to IR \to E \to R}$$

Specification defines admissible semantic possibility.

Configuration Manifold 1 represents the mandatory admissible configurations.

Configuration Manifold 2 represents the preferred admissible configurations under declared criteria.

The Instance Constitution / Instance Configuration binds the slice selected for this particular system.

The Instance Realization / Implementation Realization makes that pathway concrete.

Evidence then travels back through correspondence:

$$I \to E \to Corr(IC, I).$$

Dynatology supplies the semantics of possible continuation through these spaces.

The same representation extends to consequential execution.

The application proposes an operation:

$$O_r.$$

The constitutional system selects an admitted realization:

$$O_r \xrightarrow{\text{ACG}} R_a.$$

The executor performs an effect:

$$R_a \xrightarrow{\text{executor}} E_a.$$

Evidence then closes the loop:

$$E_a \xrightarrow{\text{witness}} Evidence.$$

This preserves a critical chain of non-collapse:

$$\begin{aligned} &\text{intent} \\ &\neq \text{operation} \\ &\neq \text{admitted realization} \\ &\neq \text{actual effect} \\ &\neq \text{evidentiary claim} \\ &\neq \text{standing} \end{aligned}.$$

The effect membrane is therefore not merely a security check. Nor is mutation itself the novelty. Its significance lies in integrating realization selection with constitutional subject binding, correspondence, evidence, receipts, authority, and standing.

Documentation then becomes another projection of this nexus rather than an independently maintained account of it.

The project can regenerate:

- semantic explanations;
- configuration references;
- current-instance orientation;
- implementation codices;
- capability maps;
- operator documentation;

from retained source representations.

Those projections remain zero-authority and do not establish realization merely by being internally consistent.

The deeper claim of this paper is therefore about **representation and reasoning**.

A reasoning-capability gain occurs when a richer representation, without changing the underlying observation, enables derivation of a decision-relevant distinction, contradiction, obligation, or restriction unavailable to the baseline representation without equivalent added structure.

The projection nexus supplies those distinctions.

A continuation may be conceivable but not admissible.

An admissible configuration may not be selected.

A selected configuration may not be realized.

A realization may occur without authorization.

An operation may be authorized only through substitution.

An effect may occur but lack adequate witness.

Evidence may exist but fail to establish a proposition.

A proposition may be established but confer no authority.

Documentation may accurately reflect the model while the implementation does not correspond.

Each distinction increases what the development system can reason about.

The resulting architecture can be summarized as:

```
agent proposes

representations type the problem

semantic constraints narrow possibility

evidence grounds observations

courts adjudicate
```

```
authority governs

effect realization is selected

executor effects

receipts preserve causality

correspondence is re-established

deterministic projections explain
```

The software-development process therefore moves beyond:

```
change
-> test
-> deploy
```

toward:

```
propose
-> specify
-> locate possibilities
-> select instance constitution
-> realize
-> propose effect
-> adjudicate
-> execute
-> witness
-> synchronize
-> preserve
-> project
```

CI/CD made testing continuous.

ACG makes adjudication continuous.

Constitutional synchronization makes correspondence continuous.

Effect interposition makes consequential execution governable and causally observable.

Deterministic projection makes explanation reproducible.

The combined shift is from software development as manipulation of implementation artifacts toward:

> **software development as governed traversal through a synchronized semantic possibility space whose consequential**

edges are constitutionally adjudicated and whose representations remain mechanically connected.

For agentic coding, that may be the more important capability upgrade.

The objective is not merely to build an agent capable of producing more code.

It is to give the agent a better world to reason about–and to ensure that even when it proposes what should happen next, a separate constitutional architecture determines what may truthfully be believed, what may legitimately be relied upon, and what may actually become real.

Appendix A. Current Realization Funnel

$$
\begin{aligned}
& P \\
& \to S \\
& \to CM_1 \\
& \to CM_2 \\
& \to IC \\
& \to IR \\
& \to E \\
& \to R
\end{aligned}
\quad .
$$

Possibility (P). Candidate problems, meanings, structures, and realizations under consideration.

Specification (S). Typed semantic possibility, invariants, operations, forbidden states, and claim boundaries.

Configuration Manifold 1 (CM1). Configurations satisfying mandatory admissibility.

Configuration Manifold 2 (CM2). Preferred admissible configurations under declared and versioned objectives and risks.

Instance Constitution / Instance Configuration (IC). The selected and bound configuration slice for a concrete system.

Instance Realization / Implementation Realization (IR). The exact concrete realization of that IC.

Evidence (E). Claim-scoped observations, methods, subjects, contexts, witnesses, and counterevidence.

Revision (R). Lineage-preserving change to any earlier layer.

Correspondence evidence can flow back from IR without collapsing observation, claim, authority, or standing:

$$IR \to E \to Corr(IC, IR) \to R?.$$

Appendix B. Core Non-Collapse Rules

$$S \neq CM$$

$$CM_1 \neq CM_2$$

$$CM_2 \neq IC$$

$$IC \neq IR$$

model ≠ realization

realization ≠ evidence

evidence ≠ admission

admission ≠ authority

authority ≠ effect

requested realization ≠ actual effect

effect ≠ standing

coverage ≠ satisfaction

identity reuse ≠ path trust

evidence reuse ≠ effect reuse

skip ≠ write

projection ≠ law

description ≠ standing

conceivable ≠ admissible

admissible ≠ selected

selected ≠ authorized

authorized ≠ realized

proposal capability ≠ effect capability.

Appendix C. Example Court Across Representation and Effect Layers

```
COURT:
  SAVE_REALIZATION_CORRESPONDENCE

Subject:
  exact application artifact / current IC

Specification:
  configurable save namespaces are admitted

Configuration Manifold:
  classic
  isolated
  temporary diagnostic
  simulated
  read-only refusal

Instance Constitution:
```

```
  selected = isolated

Proposed Operation:
  SAVE_GAME

Requested Realization:
  write legacy path

Constitutional Disposition:
  SUBSTITUTE

Admitted Realization:
  write isolated namespace

Implementation:
  filesystem effect adapter
  native save resolver

Required Evidence:
  request receipt
  disposition receipt
  actual write witness
  resolver-consumption witness
  persistence witness

Possible Dispositions:
  PASS
  NOT_ESTABLISHED
  REFUSE_CORRESPONDENCE_MISMATCH
  REFUSE_UNRECEIPTED_EFFECT

Authority Consequence:
  NONE beyond the specifically admitted save realization
```

A successful filesystem write alone does not establish save-system correspondence.

Appendix D. Projection Taxonomy

```
SPECIFICATION PROJECTION
  glossary
  semantic reference
  invariant explanation

CONFIGURATION-MANIFOLD PROJECTION
  available configurations
```

```
  realization alternatives
  exclusions / dependencies
  compatibility map

INSTANCE-CONSTITUTION PROJECTION
  current provider
  current environment
  enabled capabilities
  authority topology
  effect profile

IMPLEMENTATION PROJECTION
  implementation codex
  source bindings
  runtime pathways
  effect adapters
  executors

EPISTEMIC PROJECTION
  established
  unknown
  defeasible
  defeated
  open obligations

EFFECT PROJECTION
  proposed
  admitted
  substituted
  refused
  executed
  observed

PEDAGOGICAL PROJECTION
  README
  explain
  orient
  operator guidance
```

Each projection remains source-bound and acquires no independent authority merely by existing.

Editorial Provenance and Standing

Source, Evidence, and Standing

This is an AI-prepared successor revision candidate derived from one exact author manuscript. The integration custody manifest is `work/paper-saturation-v1/evidence/from-cicd-to-acg/integration-custody-manifest.v0.1.json`, SHA-256 `4724DE896636817C217E4B6B7E41157F4462D8C7602B208E5EFF5699A914B71F`. The exact source remains unchanged. The ACG thesis, courts model, constitutional synchronization proposal, effect-interposition architecture, deterministic-projection argument, reasoning-capability definition, development cases, research program, implications, conclusion, and original references are author material. This pass repairs source-to-Markdown projection defects, reconciles the representation with the full realization funnel, adds GMB and universal-GSM placement, maps direct corpus interfaces, and adds counterevidence and author-decision surfaces. Those additions are AI editorial or synthetic contributions until adopted. The four development episodes retain the differentiated evidentiary descriptions supplied by the author. Their underlying exact evidence packages are not bound by this manuscript and were not replayed or independently audited here. They therefore support architectural illustration and internal hypothesis formation, not independent proof of ACG effectiveness. External references, novelty, legal analogy, and cross-domain generalization were not revalidated in this internal-only pass.

Corpus-Wide Reference Positioning

This paper argues that continuous integration and continuous delivery transformed software engineering by making builds, tests, packaging, and deployment checks part of the infrastructure of change rather than optional developer practices. Its nearest external comparison fields are software, open sovereign, security. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: Modularity, secure-development practice, and reproducibility reduce classes of failure but do not guarantee architectural coherence or user sovereignty. Source availability and reproducible builds improve inspectability but do not alone guarantee comprehensibility, practical exit, hardware control, or sustainable maintenance. Unforgeable capabilities can constrain exercise of authority while still leaving allocation, delegation, revocation, and social legitimacy incorrectly specified. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] David L. Parnas. "On the Criteria To Be Used in Decomposing Systems into Modules." *Communications of the ACM* 15(12), 1972, pp. 1053-1058. DOI: 10.1145/361598.361623. https://doi.org/10.1145/361598.361623 **Relationship:** Foundational modularity treatment used to compare information hiding with explicit semantic and governance interfaces.

[**E2**] Ward Cunningham. "The WyCash Portfolio Management System." *OOPSLA '92 Experience Report*, 1992. DOI: 10.1145/157709.157715. https://doi.org/10.1145/157709.157715 **Relationship:** Original technical-debt metaphor used to distinguish implementation debt from semantic debt.

[**E3**] Karen Scarfone, Murugiah Souppaya, and Donna Dodson. *Secure Software Development Framework (SSDF) Version 1.1.* NIST SP 800-218, 2022. DOI: 10.6028/NIST.SP.800-218. https://doi.org/10.6028/NIST.SP.800-218 **Relationship:** Adjacent secure-development framework for preparation, protection, production, and vulnerability response.

[**E4**] Reproducible Builds Project. *Formal Definition of Reproducible Builds.* reproducible-builds.org source **Relationship:** Adjacent reproducibility practice; byte reproduction is compared with, but not equated to, semantic or constitutional continuity.

[**E5**] Free Software Foundation. *The Free Software Definition.* https://www.gnu.org/philosophy/free-sw.html **Relationship:** Normative predecessor for user freedoms; the paper may add reconstruction, governance, or hardware-boundary obligations beyond license rights.

[**E6**] Open Source Initiative. *The Open Source Definition.* https://opensource.org/osd **Relationship:** Adjacent licensing baseline; source availability is not treated as sufficient for operational sovereignty.

[**E7**] Martin Kleppmann et al. "Local-First Software: You Own Your Data, in Spite of the Cloud." *Onward! 2019*, pp. 154-178. DOI: 10.1145/3359591.3359737. https://www.inkandswitch.com/local-first/ **Relationship:** Adjacent local-first design program relevant to user custody, offline operation, and collaboration.

Ableman corpus

[**A1**] Adam Ableman. *Sovereign Software.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-24C6159B98023A9C`;
source SHA-256
`6AAFE79E438B980B75D1910A897F7BA716A10CF1187B68CB1F71405F36725A18`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *Constitutional Software Analysis (CSoftA): A Governance-Based Classification Method for Software Systems.* Ableman Corpus current edition; work identity `PAPER.PUB.20472196`;
source SHA-256
`3670B28B7C1AF5467BE4002C47B5F964F8FAF6F257125C2F84CE06904E4FD0DE`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *Semantic Fragmentation and Constitutional Structure in Software Systems.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-B307F65B51C503DE`;
source SHA-256
`9D2A9D79DE8BC3F2F8ABECA073E4D3607F7623F4368E63F8036081E39B4C75C3`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Governed Coding and Debugging.* Ableman Corpus current edition; work identity `PAPER.PUB.18115533`;
source SHA-256
`2469EDA96DBC1E207855A9AD27314A7FAB59FA35E6EFEF9288C30F949376BF16`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Constitutional Operating Systems.* Ableman Corpus current edition; work identity

AMCS-LOCAL-WORK-F5FB8FA82270B989;
source SHA-256
E982E4D6BD4B519EED6A7438367844954D6D5F1C2C4CC0099AAAC4AB9A00D3AB.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *Jnana*. Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-ECEAB04A620F163D;
source SHA-256
76D921D2975BDCD25D6DFB506F5E85F7AFDBF4D2CCC65F8C45394121F9D90060.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *Jnana and the Next Open Source Revolution*. Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-473AD6FAA1659E24;
source SHA-256
E73859FD3B4AD4F7ED80E96F93F2D5A0DCB3A2D76C5741813F878B3874D7887D.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Editorial source notes retained from predecessor

Beck, K. (2002). *Test Driven Development: By Example*. Addison-Wesley.

Clarke, E. M., Grumberg, O., Kroening, D., Peled, D., and Veith, H. (2018). *Model Checking* (2nd ed.). MIT Press.

de Moura, L., and Bjrner, N. (2008). Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems*. Springer.

Duvall, P. M., Matyas, S., and Glover, A. (2007). *Continuous Integration: Improving Software Quality and Reducing Risk*. Addison-Wesley.

Fowler, M. (2006). Continuous Integration.

Humble, J., and Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.

Lamport, L. (2002). *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley.

Leveson, N. G. (2011). *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press.

Adam Ableman (2026a). *Ableman's Razor: A Typed Grammar for Ontology Exploration, Configuration Collapse, and Implementation Feedback*. Manuscript.

Adam Ableman (2026b). *Legitimacy Without Self-Ratification: Constitutional Computing and the Provenance of Authority in Reflexive Systems*. Manuscript.

Adam Ableman (2026c). *Constitutional Semantic Engineering: Preserving Meaning, Authority, and Evidence Across Change*. Manuscript.

Adam Ableman (2026d). *Universal Governance Kernel: Courts, Evidence, Receipts, Continuity, and Governed Effects*. Ableman Corpus edition and system release lineage.

Newcombe, C., Rath, T., Zhang, F., Munteanu, B., Brooker, M., and Deardeuff, M. (2015). How Amazon Web Services Uses Formal Methods. *Communications of the ACM*, 58(4), 66–73.

Saltzer, J. H., and Schroeder, M. D. (1975). The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9), 1278–1308.

Schneider, F. B. (2000). Enforceable Security Policies. *ACM Transactions on Information and System Security*, 3(1), 30–50.

Weyuker, E. J. (1982). On Testing Non-Testable Programs. *The Computer Journal*, 25(4), 465–470.

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.23847750